

```
1 import pandas as pd
```

```
1 df = pd.read_csv("used_phone.csv")
```

```
1 df
```

	brand	model	ram_gb	storage_gb	condition	battery_health	age_years	original_price	resale_price
0	Samsung	Galaxy A55 5G	8	256	Like New	61	2	39999	17567
1	Oppo	OPPO A58 5G	6	128	Fair	69	1	13999	5216
2	OnePlus	OnePlus Nord N30 SE 5G	4	128	Like New	69	1	13999	7824
3	Redmi	POCO X5 Pro 5G	6	128	Fair	87	3	20999	7673
4	Realme	Realme Narzo 60x 5G	6	128	Fair	82	1	12999	5755
...	...	...	...	...	...	...	...	...	...
4995	Motorola	Motorola Edge 30 Ultra	12	256	Fair	77	1	49999	20789
4996	OnePlus	OnePlus 10R 5G	8	128	Fair	55	4	32999	6533
4997	Vivo	Vivo Y16	3	64	Like New	88	3	8999	4989
4998	Redmi	Redmi A3	3	64	Good	76	2	6999	3191
4999	Apple	iPhone 15 Pro (512GB)	8	512	Fair	84	2	149900	60439

5000 rows × 9 columns

```
1 df.shape
```

```
(5000, 9)
```

```
1 df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype  
---  -
0   brand           5000 non-null   object 
1   model           5000 non-null   object 
2   ram_gb          5000 non-null   int64  
3   storage_gb      5000 non-null   int64  
4   condition       5000 non-null   object 
5   battery_health  5000 non-null   int64  
```

```
6  age_years      5000 non-null  int64
7  original_price  5000 non-null  int64
8  resale_price   5000 non-null  int64
dtypes: int64(6), object(3)
memory usage: 351.7+ KB
```

```
1 df.isnull().sum()
```

	0
brand	0
model	0
ram_gb	0
storage_gb	0
condition	0
battery_health	0
age_years	0
original_price	0
resale_price	0

dtype: int64

```
1 df.duplicated().sum()
```

np.int64(99)

```
1 df = df.drop_duplicates()
2 df = df.reset_index(drop=True) # 0,1,2,3,4,5
```

```
1 df.shape
```

(4901, 9)

```
1 df.duplicated().sum()
```

np.int64(0)

```
1 df.head()
```

	brand	model	ram_gb	storage_gb	condition	battery_health	age_years	original_price	resale_price
0	Samsung	Galaxy A55 5G	8	256	Like New	61	2	39999	17567
1	Oppo	OPPO A58 5G	6	128	Fair	69	1	13999	5216
2	OnePlus	OnePlus Nord N30 SE 5G	4	128	Like New	69	1	13999	7824
3	Redmi	POCO X5 Pro 5G	6	128	Fair	87	3	20999	7673
4	Realme	Realme Narzo 60x 5G	6	128	Fair	82	1	12999	5755

✓ object data converted into numerical with sklearn labelencoder

1. brand
2. model
3. condition

```
1 from sklearn.preprocessing import LabelEncoder
```

```
1 le_brand = LabelEncoder() # instead
2 le_model = LabelEncoder()
3 le_condition = LabelEncoder()
```

```
1 df["brand"] = le_brand.fit_transform(df["brand"])
2 df["model"] = le_model.fit_transform(df["model"])
3 df["condition"] = le_condition.fit_transform(df["condition"])
```

```
1 df['brand']
```

	brand
--	-------

0	7
1	4
2	3
3	6
4	5
...	...
4896	2
4897	3
4898	8
4899	6
4900	0

4901 rows × 1 columns

dtype: int64

```
1 df["model"]
```

model	
0	8
1	54
2	79
3	84
4	103
...	...
4896	27
4897	70
4898	127
4899	111
4900	161

4901 rows × 1 columns

dtype: int64

```
1 df["condition"]
```

condition

0	2
1	0
2	2
3	0
4	0
...	...
4896	0
4897	0
4898	2
4899	1
4900	0

4901 rows × 1 columns

dtype: int64

1 df.head()

	brand	model	ram_gb	storage_gb	condition	battery_health	age_years	original_price	resale_price
0	7	8	8	256	2	61	2	39999	17567
1	4	54	6	128	0	69	1	13999	5216
2	3	79	4	128	2	69	1	13999	7824
3	6	84	6	128	0	87	3	20999	7673
4	5	103	6	128	0	82	1	12999	5755

1 df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4901 entries, 0 to 4900
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   brand           4901 non-null  int64
1   model           4901 non-null  int64
2   ram_gb          4901 non-null  int64
3   storage_gb      4901 non-null  int64
4   condition       4901 non-null  int64
```

```
5  battery_health  4901 non-null  int64
6  age_years      4901 non-null  int64
7  original_price  4901 non-null  int64
8  resale_price   4901 non-null  int64
dtypes: int64(9)
memory usage: 344.7 KB
```

✓ Divided into independent & depending variable

```
1 x = df.drop("resale_price", axis=1)
2 y = df["resale_price"]
```

```
1 x.head(2)
```

	brand	model	ram_gb	storage_gb	condition	battery_health	age_years	original_price
0	7	8	8	256	2	61	2	39999
1	4	54	6	128	0	69	1	13999

```
1 y.head(2)
```

	resale_price
0	17567
1	5216

dtype: int64

✓ Train Test split

```
1 from sklearn.model_selection import train_test_split
2 x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
```

```
1 print("The x train shape : ", x_train.shape)
2 print("The x test shape  : ", x_test.shape)
3 print("The y train shape : ", y_train.shape)
4 print("The y test shape  : ", y_test.shape)
```

```
The x train shape : (3920, 8)
The x test shape  : (981, 8)
The y train shape : (3920,)
The y test shape  : (981,)
```

✓ Training The Model Random Forest Regressor

```
1 from sklearn.ensemble import RandomForestRegressor
```

```
1 model = RandomForestRegressor(
2     n_estimators=100,
3     random_state=42,
4 )
```

```
1 model.fit(x_train, y_train)
```

▼ RandomForestRegressor ⓘ ?

```
RandomForestRegressor(random_state=42)
```

```
1 y_pred = model.predict(x_test)
2 y_pred
```

[Show hidden output](#)

✓ Performance Matrix

```
1 from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
2 import numpy as np
3 r2 = r2_score(y_test, y_pred)
4 mae = mean_absolute_error(y_test, y_pred)
5 mse = mean_squared_error(y_test, y_pred)
6 rmse = np.sqrt(mse)
7
8 print("The R2 Score : ", r2)
9 print("The MAE Score : ", mae)
```

```
10 print("The MSE Score : ", mse)
```

```
The R2 Score : 0.9872939706458711
The MAE Score : 758.3131090723751
The MSE Score : 3051339.050432416
The RMSE Score : 1746.808246612208
```

```
1 df.head()
```

	brand	model	ram_gb	storage_gb	condition	battery_health	age_years	original_price	resale_price
0	7	8	8	256	2	61	2	39999	17567
1	4	54	6	128	0	69	1	13999	5216
2	3	79	4	128	2	69	1	13999	7824
3	6	84	6	128	0	87	3	20999	7673
4	5	103	6	128	0	82	1	12999	5755

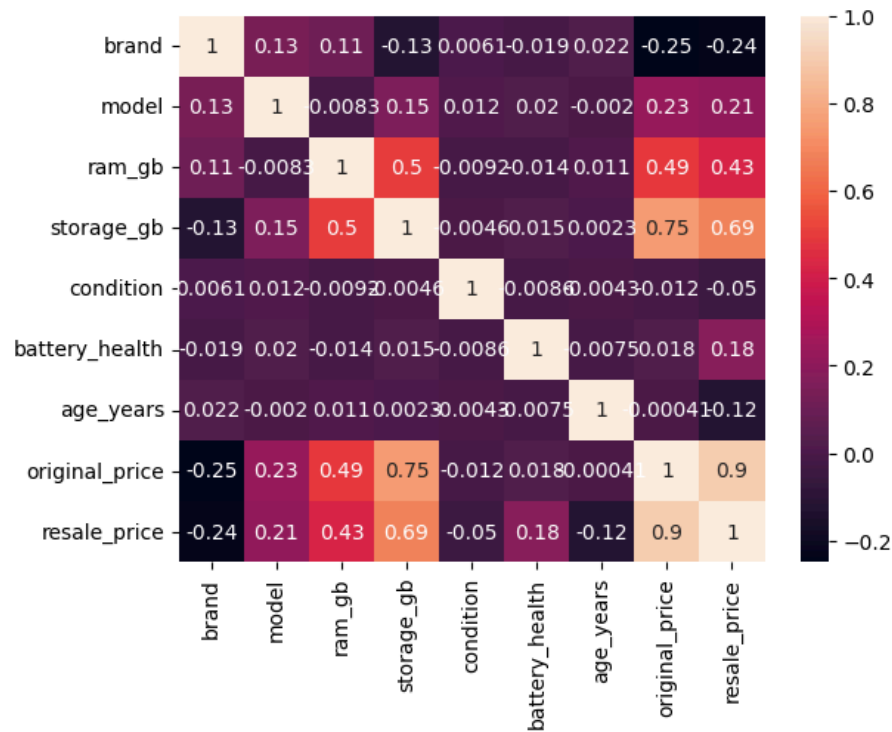
```
1 model.predict([[7,8,8,256,2,61,2,39999 ]])
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but RandomForestRegressor was fitted with fe
warnings.warn(
array([17398.04])
```

```
1 # Affect resale_price based on data
2 import seaborn as sns
3 sns.pairplot(df)
```

[Show hidden output](#)

```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
3 sns.heatmap(df.corr(), annot=True)
4 plt.show()
```



✖ pickle

```
1 import pickle
2 with open("model_phone.pkl", "wb") as f:
3     pickle.dump(model, f)
```

1 Start coding or [generate](#) with AI.

1 Start coding or [generate](#) with AI.

✖ vs code py file streamlit

```
1 import streamlit as st
2 import pandas as pd
3 import pickle
4
5 # Load the model
6 model = pickle.load(open('model_phone.pkl', 'rb'))
7
8
9 df = pd.read_csv("used_phone.csv")
10
11
12 brand_model_map = df.groupby("brand")["model"].unique().to_dict()
13
14 brand_list = list(brand_model_map.keys())
15
16 st.title("phone price prediction 📞💰")
17
18 st.sidebar.header("Enter phone Details")
19
20 selected_brand = st.sidebar.selectbox("Select Brand", brand_list)
21
22 selected_model = st.sidebar.selectbox("Select Model", brand_model_map[selected_brand])
23
24 ram = st.sidebar.selectbox("Ram(GB)", sorted(df["ram_gb"].unique()))
25
26 storage = st.sidebar.selectbox("Storage(GB)", sorted(df["storage_gb"].unique()))
27
28 condition = st.sidebar.selectbox("Condition", sorted(df["condition"].unique()))
29
30 battery_health = st.sidebar.slider("Battery Health(%)", 50, 100, 80)
31
32 age = st.sidebar.slider("Age of the phone(years)", 0, 10, 2)
33
34 original_price = st.sidebar.number_input("Original Price($)", min_value=5000, max_value=200000, value=15000)
35
36 from sklearn.preprocessing import LabelEncoder
37
38 le_brand = LabelEncoder() # instead
```

```

39 le_model = LabelEncoder()
40 le_condition = LabelEncoder()
41
42 df["brand"] = le_brand.fit_transform(df["brand"])
43 df["model"] = le_model.fit_transform(df["model"])
44 df["condition"] = le_condition.fit_transform(df["condition"])
45
46 brand_encoded = le_brand.transform([selected_brand])[0]
47 model_encoded = le_model.transform([selected_model])[0]
48 condition_encoded = le_condition.transform([condition])[0]
49
50
51 input_data = pd.DataFrame({
52     "brand": [brand_encoded],
53     "model": [model_encoded],
54     "ram_gb": [ram],
55     "storage_gb": [storage],
56     "condition": [condition_encoded],
57     "battery_health": [battery_health],
58     "age_years": [age],
59     "original_price": [original_price]
60 })
61
62 if st.sidebar.button("Predict Price"):
63     predicted_price = model.predict(input_data)[0]
64     st.success(f"Estimated Resale Price: 🏠 {int(predicted_price):,}")
65
66
67

```

↙ vs code app.py file

```

1 import streamlit as st
2 import pandas as pd
3 import pickle
4 from sklearn.preprocessing import LabelEncoder
5
6 # PAGE CONTENT

```

```

6 # --- PAGE CONFIG ---
7 st.set_page_config(page_title="Phone Price Predictor", page_icon="📱", layout="wide")
8
9 # --- CUSTOM CSS ---
10 st.markdown("""
11     <style>
12     .main {
13         background-color: #f8f9fa;
14     }
15     .stButton>button {
16         width: 100%;
17         border-radius: 12px;
18         height: 3.5em;
19         background: linear-gradient(45deg, #ff4b4b, #ff7676);
20         color: white;
21         font-weight: bold;
22         border: none;
23         transition: 0.3s;
24     }
25     .stButton>button:hover {
26         transform: scale(1.02);
27         box-shadow: 0 5px 15px rgba(255, 75, 75, 0.4);
28     }
29     .prediction-card {
30         padding: 30px;
31         background-color: white;
32         border-radius: 20px;
33         box-shadow: 0 10px 25px rgba(0,0,0,0.05);
34         text-align: center;
35         border: 1px solid #eee;
36         margin-top: 20px;
37     }
38     .price-text {
39         color: #ff4b4b;
40         font-size: 3rem;
41         font-weight: 800;
42         margin: 10px 0;
43     }
44     </style>
45 """)

```

```

45         , unsafe_allow_html=True)
46
47 # --- LOAD DATA & MODEL ---
48 @st.cache_resource
49 def load_assets():
50     # Make sure these files are in your project folder!
51     model = pickle.load(open('model_phone.pkl', 'rb'))
52     df = pd.read_csv("used_phone.csv")
53     return model, df
54
55 model, df = load_assets()
56
57 # Preparation for Selectboxes
58 brand_model_map = df.groupby("brand")["model"].unique().to_dict()
59 brand_list = sorted(list(brand_model_map.keys()))
60
61 # --- UI LAYOUT ---
62 st.title("📱 Used Phone Value Estimator")
63 st.markdown("Enter the device details below to get an instant AI-powered valuation.")
64 st.write("---")
65
66 # Main Container for inputs
67 col1, col2 = st.columns(2, gap="large")
68
69 with col1:
70     st.subheader("📋 Identity")
71     selected_brand = st.selectbox("Brand Name", brand_list)
72     selected_model = st.selectbox("Model Name", sorted(brand_model_map[selected_brand]))
73     condition = st.selectbox("Physical Condition", options=sorted(df["condition"].unique()))
74     age = st.selectbox("Age of Device (Years)", options=sorted(df["age_years"].unique()))
75
76 with col2:
77     st.subheader("⚙️ Technical Specs")
78     ram = st.selectbox("RAM Capacity (GB)", options=sorted(df["ram_gb"].unique()))
79     storage = st.selectbox("Storage Capacity (GB)", options=sorted(df["storage_gb"].unique()))

```